

Summary of Change - Matchmaking/Leaderboard

We introduced an enum for game types, which was not originally planned. Up until last week, the game type was specified using an array of numbers, which initially worked well. However, maintaining a consistent format across the codebase became increasingly difficult. To address this, we decided to implement an enum for gameType to improve readability and maintainability.

Before:

```
*/
public class Player {
    private final double[] winRatio = new double[3]; // 1: TicTacToe, 2: Connect4, 3: Checkers
    private final int[] wins = new int[3];
    private final int[] losses = new int[3];
    private int level;
    private int userID;
    private boolean spectate;
    private final Rank[] rank = new Rank[3];
    private final int[] gameSignal = new int[3];
    private final int[] mmr = new int[3];

    /**
     * Constructs a Player object
     * @param level rank level of player
     * @param userID userID of the player from database
     * @param spectate whether player is spectating
     * @param rank rank of the player
     * @param gameSignal which game the player wants to play
     */
    public Player(double winRatioForGame, int level, int userID, boolean spectate, Rank rank, int gameSignal) {
        this.level = level;
        this.userID = userID;
        this.spectate = spectate;
        for(int i = 0; i < 3; i++) {
            this.winRatio[i] = 0.0;
            this.wins[i] = 0;
            this.losses[i] = 0;
            this.rank[i] = new Rank();
            this.gameSignal[i] = 0;
            this.mmr[i] = 0;
        }
    }
}
```

Now:

```
package MatchmakingLeaderboard;

/**
 * Enum for different game types within the matchmaking leaderboard system and other places
 */
public enum GameType {
    TIC_TAC_TOE(1),
    CONNECT_FOUR(2),
    CHECKERS(3);

    private final int gameCode;

    GameType(int gameCode) {
        this.gameCode = gameCode;
    }

    public int getGameCode() {
        return this.gameCode;
    }
}
```

One of the reviewing team members recommended implementing an ELO rating system for player rankings. After careful consideration, we chose to implement a **Matchmaking Rating (MMR)** system instead. This decision was based on MMR's flexibility and its ability to account for individual performance metrics beyond simple win/loss outcomes.

Our implementation uses individual win ratios as a proxy for player skill and dynamically adjusts MMR based on:

- Match outcome (win/loss/draw)
- Relative performance between players
- Game type (TicTacToe, Connect4, or Checkers)

We also updated the planning documents, added a proper timeline to track and manage work, and included details about the classes implemented for the Matchmaking-Leaderboard system. A brief overview of the ranking system and its functionality was also provided to help ensure clarity and alignment within the team.